

Generally, people carry their favorite songs on smartphones. Songs can be of various genres. With the help of deep learning techniques, we can provide a classified list of songs to the smartphone user. We will apply deep learning algorithms to create models, which can classify audio files into various genres. After training the model, we will also analyze the performance of our trained model.

## Dataset:

We will use [GITZAN dataset](#), which contains 1000 music files. Dataset has ten types of genres with uniform distribution. Dataset has the following genres: blues, classical, country, disco, hiphop, jazz, reggae, rock, metal, and pop. Each music file is 30 seconds long.

## Process Flow:

Figure 01 represents the overview of our methodology for the genre classification task. We will discuss each phase in detail. We train three types of deep learning models to explore and gain insights from the data.

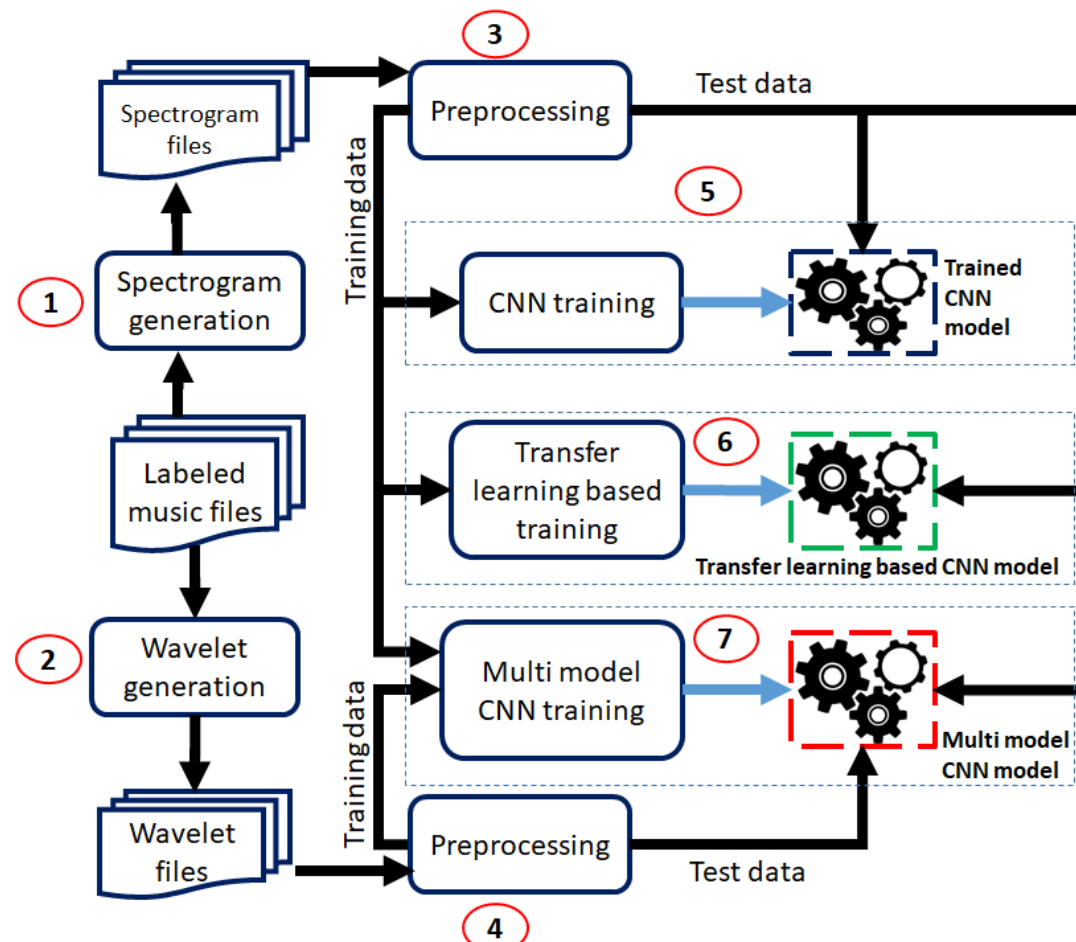


Fig. 01

First, we need to convert the audio signals into a deep learning model compatible format. We use two types of formats, which are as follows:

### 1. Spectrogram generation:

A spectrogram is a visual representation of the spectrum signal frequencies as it varies with time. We use librosa library to transform each audio file into

a spectrogram. Figure 02 shows spectrogram images for each type of music genre.

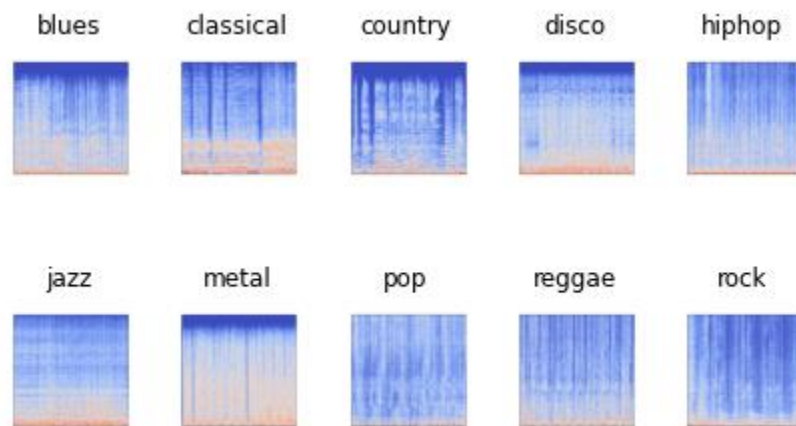


Fig. 02

## 2. Wavelet generation: –

The Wavelet Transform is a transformation that can be used to analyze the spectral and temporal properties of non-stationary signals like audio. We use librosa library to generate wavelets of each audio file. Figure 03 shows wavelets of each type of music genre.

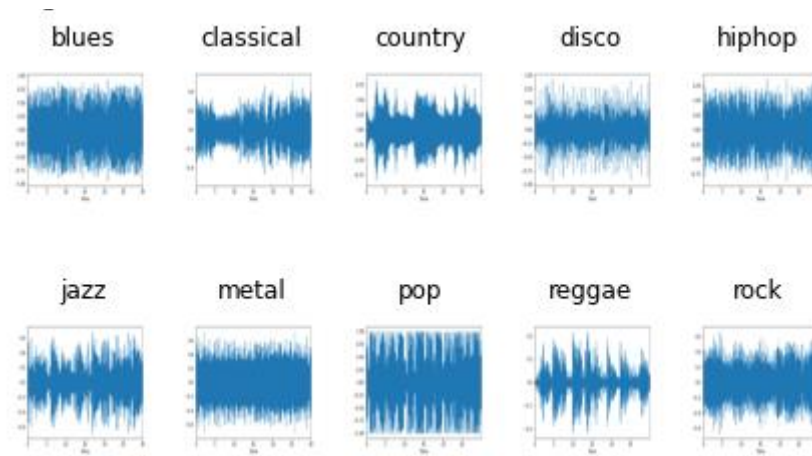


Fig. 03

### 3, 4. Spectrogram and Wavelet preprocessing

From Figure 02 and 03, it is clear that we treat our data as image data. After generating spectrograms and wavelets, we apply general image preprocessing steps to generate training and testing data. Each image is of size (256, 256, 3).

### 5. Basic CNN model training:

After preprocessing the data, we create our first deep learning model. We construct a Convolution Neural Network model with required input and out units. The final architecture of our CNN model is shown in Figure 04. We use only spectrogram data for the training and testing.

| Model: "sequential"            |                      |         |
|--------------------------------|----------------------|---------|
| Layer (type)                   | Output Shape         | Param # |
| =====                          |                      |         |
| conv2d (Conv2D)                | (None, 256, 256, 32) | 896     |
| max_pooling2d (MaxPooling2D)   | (None, 128, 128, 32) | 0       |
| conv2d_1 (Conv2D)              | (None, 128, 128, 32) | 9248    |
| max_pooling2d_1 (MaxPooling2D) | (None, 64, 64, 32)   | 0       |
| conv2d_2 (Conv2D)              | (None, 64, 64, 64)   | 18496   |
| max_pooling2d_2 (MaxPooling2D) | (None, 32, 32, 64)   | 0       |
| dropout (Dropout)              | (None, 32, 32, 64)   | 0       |
| flatten (Flatten)              | (None, 65536)        | 0       |
| dense (Dense)                  | (None, 128)          | 8388736 |
| dense_1 (Dense)                | (None, 10)           | 1290    |
| =====                          |                      |         |
| Total params: 8,418,666        |                      |         |
| Trainable params: 8,418,666    |                      |         |
| Non-trainable params: 0        |                      |         |

Fig. 04

We train our CNN model for 500 epochs with Adam optimizer at a learning rate of 0.0001. We use categorical cross-entropy as the loss function. Figure 05 shows the training and validation losses and model performance in terms of accuracy.

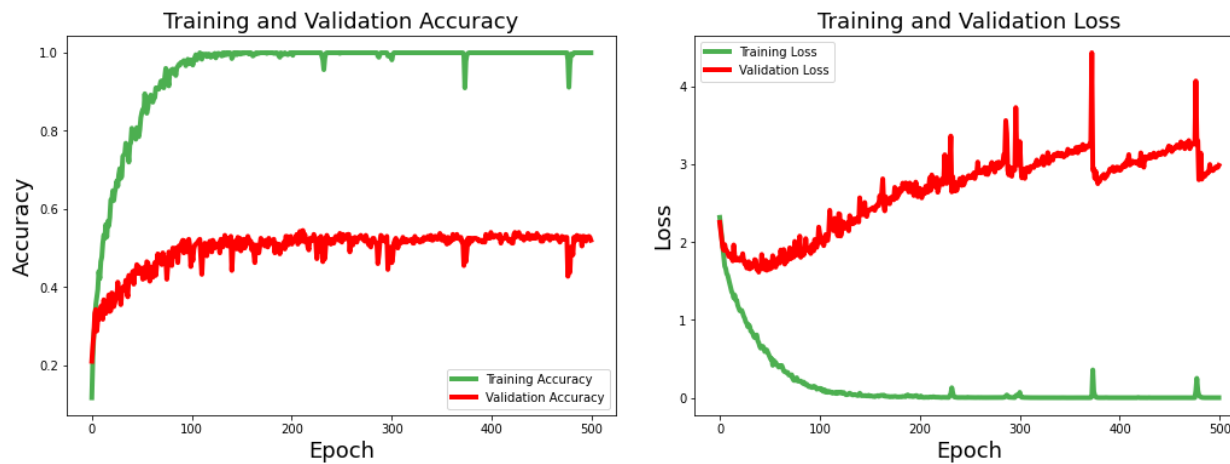


Fig. 05

## 6. Transfer learning-based model training

We have only 60 samples of each genre for training. In this case, transfer learning could be a useful option to improve the performance of our CNN model. Now, we use the pre-trained mobilenet model to train the CNN model. A schematic architecture is shown in Figure 06.

Model: "sequential"

| Layer (type)                    | Output Shape       | Param # |
|---------------------------------|--------------------|---------|
| mobilenetv2_1.00_224 (Functi    | (None, 8, 8, 1280) | 2257984 |
| global_average_pooling2d (Gl    | (None, 1280)       | 0       |
| dropout (Dropout)               | (None, 1280)       | 0       |
| dense (Dense)                   | (None, 10)         | 12810   |
| Total params: 2,270,794         |                    |         |
| Trainable params: 12,810        |                    |         |
| Non-trainable params: 2,257,984 |                    |         |

Fig. 06

The transfer learning-based model is trained with the same settings as used in the previous model. Figure 07 shows the training and validation loss and model performance in terms of accuracy. Here, also we use only spectrogram data for the training and testing.

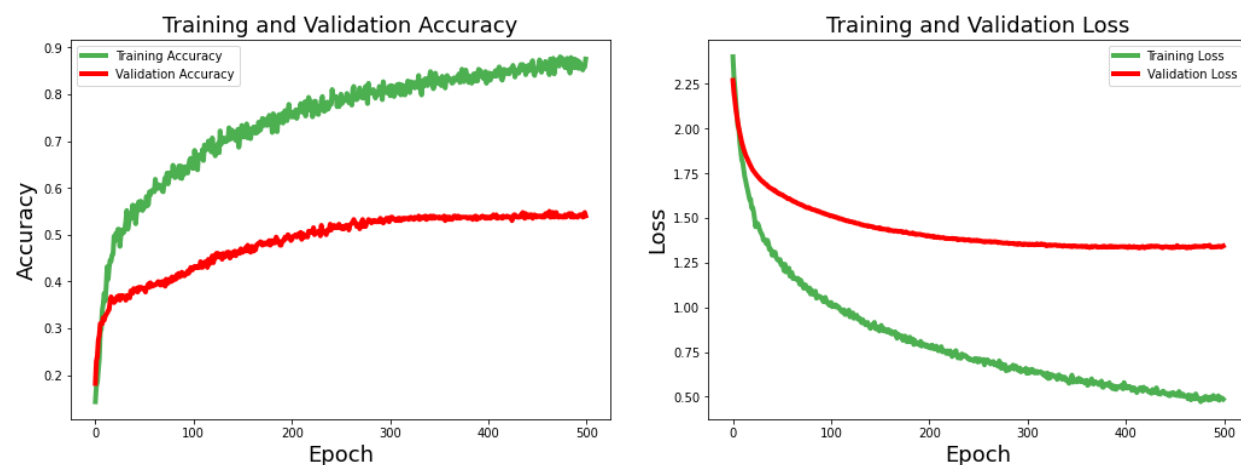


Fig. 07

## 7. Multimodal training

We will pass both spectrogram and wavelet data into the CNN model for the training in this experiment. We are using the late-fusion technique in this multi-modal training. Figure 08 represents the architecture of our multi-modal CNN model. Figure 09 shows the loss and performance scores of the model with respect to epochs.

| Layer (type)                   | Output Shape               | Param # | Connected to                 |
|--------------------------------|----------------------------|---------|------------------------------|
| input_1 (InputLayer)           | [(None, 256, 256, 3)] 0    |         |                              |
| input_2 (InputLayer)           | [(None, 256, 256, 3)] 0    |         |                              |
| conv2d (Conv2D)                | (None, 256, 256, 32) 896   |         | input_1[0][0]                |
| conv2d_2 (Conv2D)              | (None, 256, 256, 32) 896   |         | input_2[0][0]                |
| max_pooling2d (MaxPooling2D)   | (None, 128, 128, 32) 0     |         | conv2d[0][0]                 |
| max_pooling2d_2 (MaxPooling2D) | (None, 128, 128, 32) 0     |         | conv2d_2[0][0]               |
| conv2d_1 (Conv2D)              | (None, 128, 128, 64) 18496 |         | max_pooling2d[0][0]          |
| conv2d_3 (Conv2D)              | (None, 128, 128, 64) 18496 |         | max_pooling2d_2[0][0]        |
| max_pooling2d_1 (MaxPooling2D) | (None, 64, 64, 64) 0       |         | conv2d_1[0][0]               |
| max_pooling2d_3 (MaxPooling2D) | (None, 64, 64, 64) 0       |         | conv2d_3[0][0]               |
| dropout (Dropout)              | (None, 64, 64, 64) 0       |         | max_pooling2d_1[0][0]        |
| dropout_1 (Dropout)            | (None, 64, 64, 64) 0       |         | max_pooling2d_3[0][0]        |
| flatten (Flatten)              | (None, 262144) 0           |         | dropout[0][0]                |
| flatten_1 (Flatten)            | (None, 262144) 0           |         | dropout_1[0][0]              |
| dense (Dense)                  | (None, 128) 33554560       |         | flatten[0][0]                |
| dense_1 (Dense)                | (None, 128) 33554560       |         | flatten_1[0][0]              |
| tf.concat (TFOpLambda)         | (None, 256) 0              |         | dense[0][0]<br>dense_1[0][0] |
| dense_2 (Dense)                | (None, 32) 8224            |         | tf.concat[0][0]              |
| dense_3 (Dense)                | (None, 10) 330             |         | dense_2[0][0]                |
| Total params: 67,156,458       |                            |         |                              |
| Trainable params: 67,156,458   |                            |         |                              |
| Non-trainable params: 0        |                            |         |                              |

Fig. 08

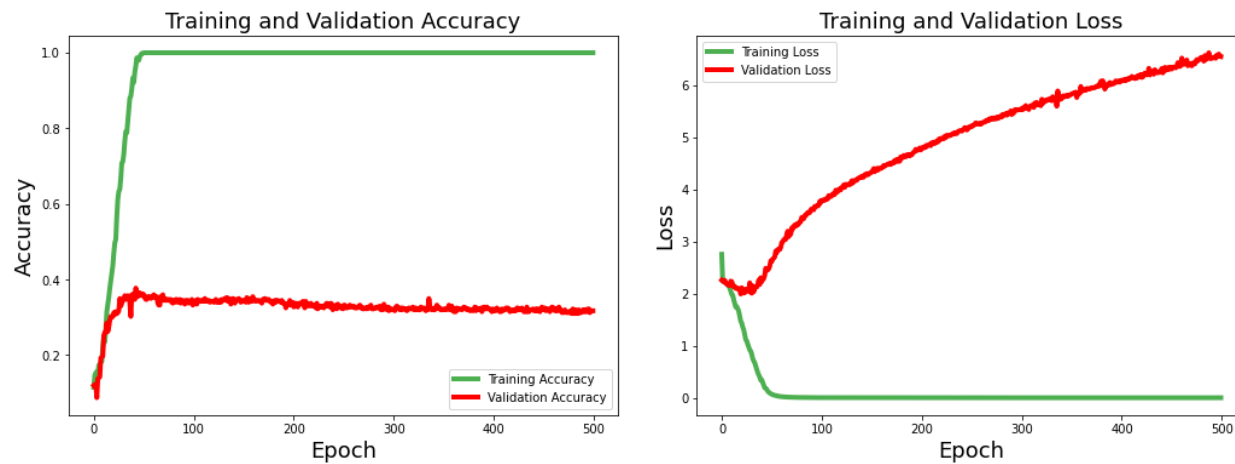


Fig. 09

## Comparison:

Figure 10 shows a comparative analysis of the loss and performance of all three models. If we analyze the training behavior of all three models, we found that the basic CNN model has large fluctuations in its loss values and performance scores for training and testing data. The multimodal model has shown the least variance in performance. Transfer learning model performance increases gradually compared to multimodal and basic CNN models. Validation loss value shot up suddenly after the 30 epochs. On the other side, validation loss decreases continuously for the other two models.

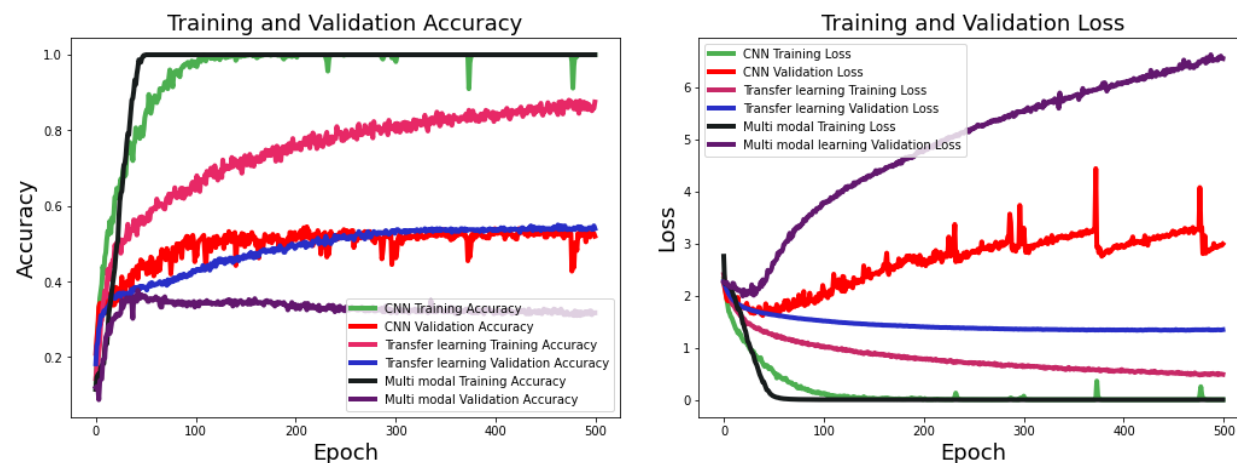


Fig. 10

## Testing the models

After training our models, we test each model on the 40% test data. We calculate precision, recall, and F-score for each music genre (class). Our dataset is balanced; therefore, the macro average and weighted average of precision, recall, and F-score are the same.

### 1. Basic CNN model

Figure 11 presents the results of our CNN model on the test data. CNN model was able to classify “classical” genre music with the highest F1-score. CNN performed worst for “Rock” and “reggae” genre music. Figure 12 shows the confusion matrix of the CNN model on the test data.

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| blues        | 0.46      | 0.55   | 0.50     | 40      |
| classical    | 1.00      | 0.80   | 0.89     | 40      |
| country      | 0.42      | 0.42   | 0.42     | 40      |
| disco        | 0.40      | 0.45   | 0.42     | 40      |
| hiphop       | 0.41      | 0.60   | 0.49     | 40      |
| jazz         | 0.57      | 0.50   | 0.53     | 40      |
| metal        | 0.81      | 0.72   | 0.76     | 40      |
| pop          | 0.72      | 0.53   | 0.61     | 40      |
| reggae       | 0.37      | 0.38   | 0.37     | 40      |
| rock         | 0.28      | 0.25   | 0.26     | 40      |
| accuracy     |           |        | 0.52     | 400     |
| macro avg    | 0.54      | 0.52   | 0.53     | 400     |
| weighted avg | 0.54      | 0.52   | 0.53     | 400     |

Fig. 11

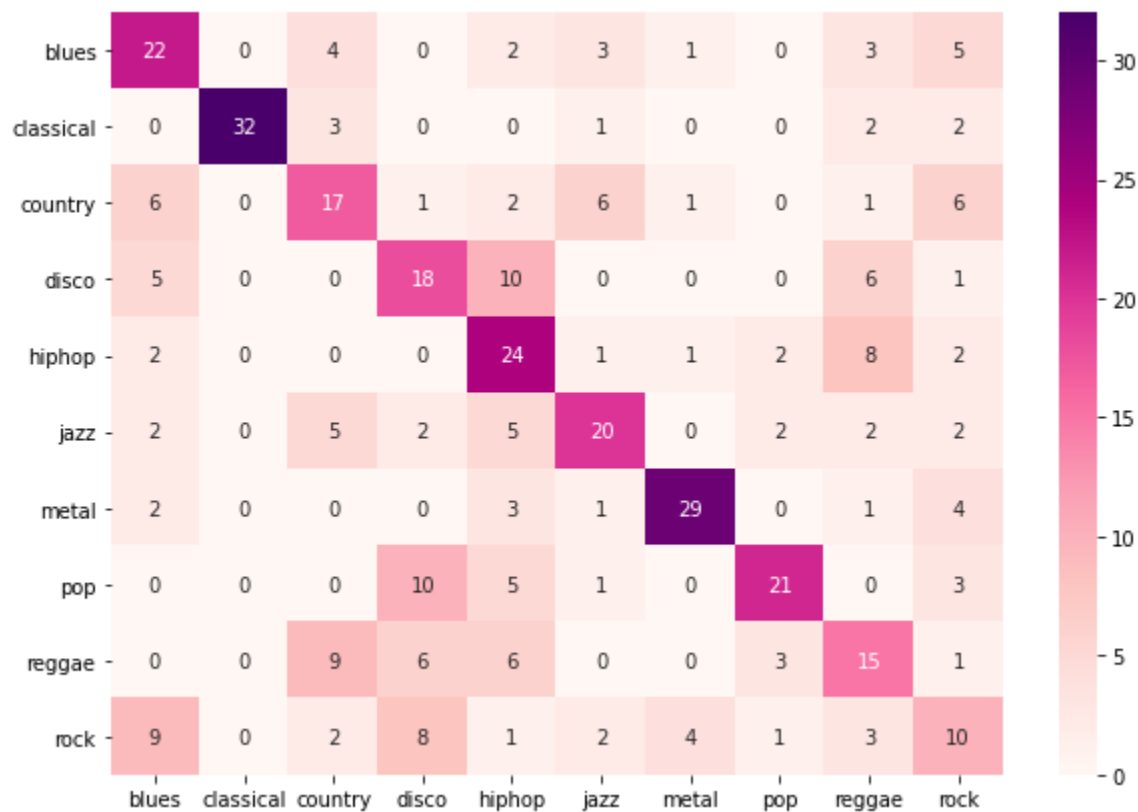


Fig. 12

## 2. Transfer learning based model

We used the transfer learning technique to improve the performance of genre classification. Figure 13 presents the results of the transfer learning-based model on test data. F1-score for “hiphop”, “jazz”, and “pop” genres increased due to transfer learning. If we look at overall results, we have achieved only a minor improvement after applying transfer learning. Figure 14 shows the confusion matrix for the transfer learning model on the test data.

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| blues        | 0.43      | 0.57   | 0.49     | 40      |
| classical    | 0.90      | 0.88   | 0.89     | 40      |
| country      | 0.43      | 0.33   | 0.37     | 40      |
| disco        | 0.46      | 0.45   | 0.46     | 40      |
| hiphop       | 0.43      | 0.72   | 0.54     | 40      |
| jazz         | 0.81      | 0.65   | 0.72     | 40      |
| metal        | 0.67      | 0.60   | 0.63     | 40      |
| pop          | 0.84      | 0.65   | 0.73     | 40      |
| reggae       | 0.35      | 0.33   | 0.34     | 40      |
| rock         | 0.26      | 0.23   | 0.24     | 40      |
| accuracy     |           |        | 0.54     | 400     |
| macro avg    | 0.56      | 0.54   | 0.54     | 400     |
| weighted avg | 0.56      | 0.54   | 0.54     | 400     |

Fig. 13

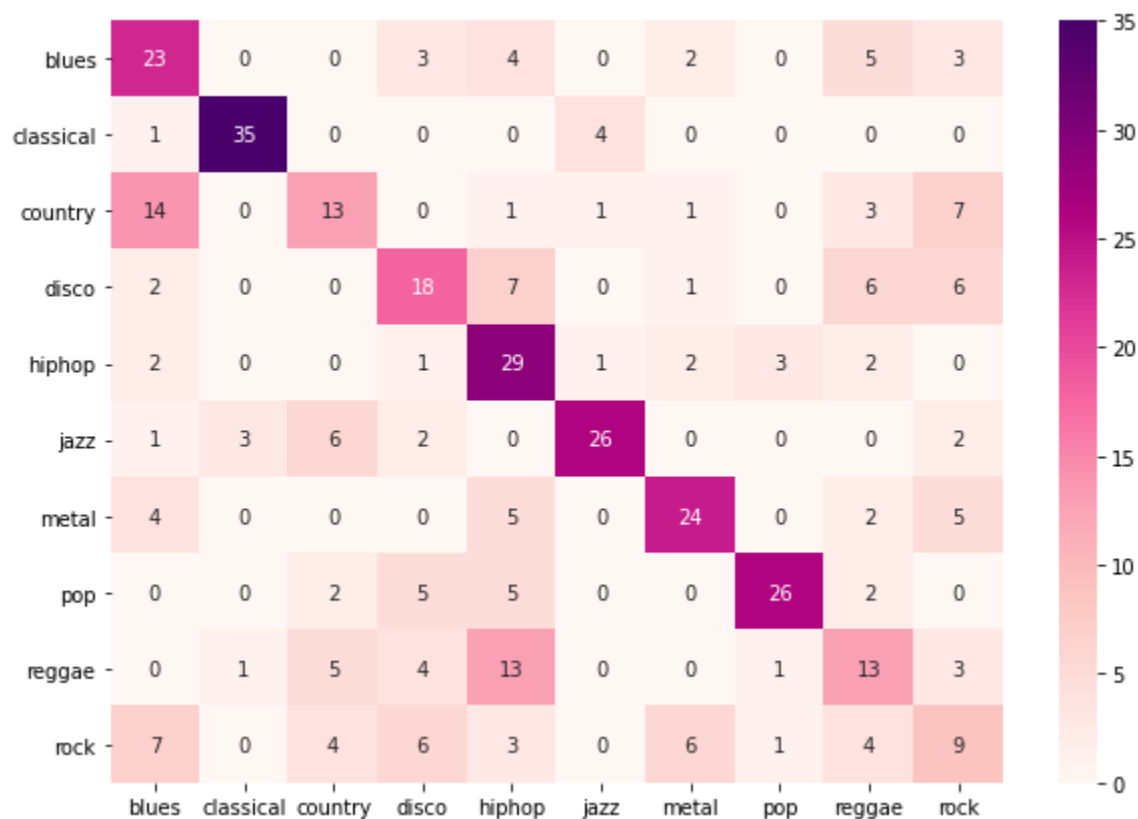


Fig. 14

3. Multimodal-based model: We have used both spectrogram and wavelet data to train the multimodal-based model. In the same way, we perform the testing. We have found very surprising results. Instead of improvement, our performance reduced drastically. We have achieved only 38% of F1-score while using a multi-modal approach. Figure 16 shows the confusion matrix of the multimodal-based model on the test data.

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| blues        | 0.27      | 0.25   | 0.26     | 40      |
| classical    | 0.83      | 0.72   | 0.77     | 40      |
| country      | 0.20      | 0.23   | 0.21     | 40      |
| disco        | 0.34      | 0.25   | 0.29     | 40      |
| hiphop       | 0.29      | 0.50   | 0.37     | 40      |
| jazz         | 0.38      | 0.23   | 0.28     | 40      |
| metal        | 0.51      | 0.62   | 0.56     | 40      |
| pop          | 0.71      | 0.55   | 0.62     | 40      |
| reggae       | 0.24      | 0.23   | 0.23     | 40      |
| rock         | 0.20      | 0.23   | 0.21     | 40      |
| accuracy     |           |        | 0.38     | 400     |
| macro avg    | 0.40      | 0.38   | 0.38     | 400     |
| weighted avg | 0.40      | 0.38   | 0.38     | 400     |

Fig. 15

